Inside Stripe's Rate Limiter Architecture

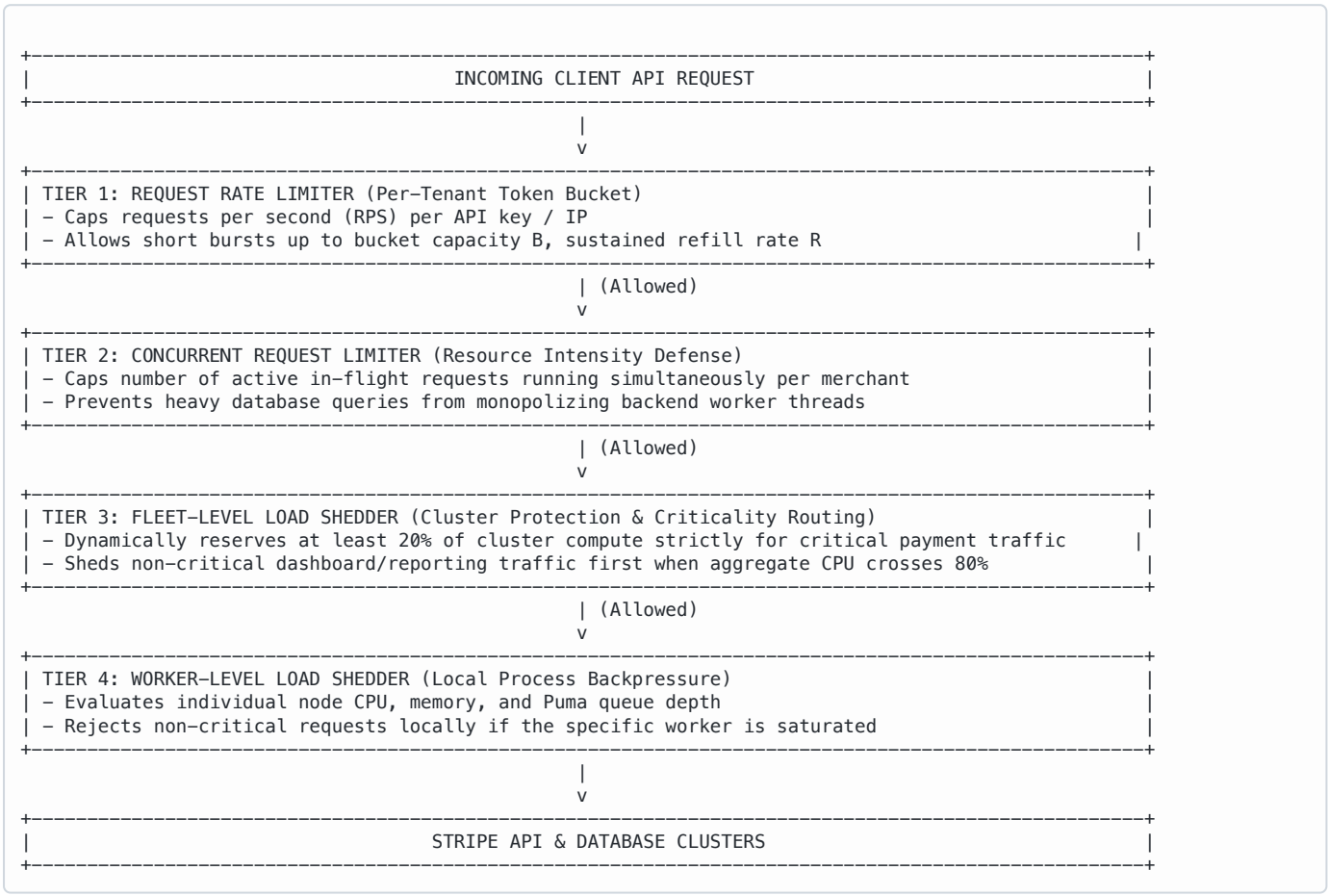
Source: Arpit Bhayani Dissection • Stripe Infrastructure
Focus: Four Tiers: Rate Limiters & Load Shedders

1. The Financial Scale Challenge

Operating payment infrastructure requires exceptional availability: a dropped payment request during high-traffic retail events directly translates into lost merchant revenue. However, public APIs are constantly vulnerable to buggy client loops, card-testing fraud bots, and heavy analytical reporting queries that can starve critical transaction threads.

A naive single-point rate limiter (e.g. 100 requests/minute per key) is insufficient. To balance tenant fairness, prevent resource exhaustion, and guarantee that critical checkout transactions always succeed, Stripe deployed a **four-tier defensive architecture** comprising two tenant-level rate limiters and two infrastructure-level load shedders.

2. Architecture of the Four Defensive Tiers



Defense-in-Depth Axiom: Rate limiters protect against misbehaving clients; load shedders protect against aggregate system overload. By separating tenant volume from system health, critical payments process smoothly even during cluster stress.

Rate Limiting Algorithms & Concurrency Guards

Source: Arpit Bhayani Dissection • Stripe Infrastructure
Focus: Token Bucket Formalism, Concurrency Tracking & Redis

3. Tier 1: Token Bucket Request Limiter

The request rate limiter operates on a per-tenant key (e.g., merchant ID) using a **Token Bucket** algorithm backed by distributed Redis clusters:

- A bucket holds up to B tokens. Every incoming request consumes 1 token.
- Tokens refill continuously at rate R tokens per second based on elapsed time: $$ ext{Tokens}_{ ext{now}} = \min(B, T_{ ext{last}} + (t_{ ext{now}} - t_{ ext{last}}) \cdot R)$$
- If $ ext{Tokens}_{ ext{now}} \ge 1$, the request is admitted and $ ext{Tokens}_{ ext{now}}$ is decremented. If $ ext{Tokens}_{ ext{now}} < 1$, the request is throttled with HTTP 429.

4. Tier 2: Concurrent Request Limiter

Measuring requests-per-second is inadequate for heavy operations. Ten fast point-lookups execute in milliseconds, but ten complex reporting queries (`GET /v1/charges?created[gte]=...`) can monopolize database connections for seconds.

```
Merchant A sends 20 concurrent queries:
Req 1  ==========[ Processing DB Query (450ms) ]==========> Done
Req 2  ==========[ Processing DB Query (510ms) ]==========> Done
Req 3  ==========[ Processing DB Query (480ms) ]==========> Done
...
Req 15 ==========[ Processing DB Query (500ms) ]==========> Done
-------------------------------------------------------------------------
Req 16+ ===> [ Concurrent Limiter: Active >= 15 ] ===> HTTP 429 (Throttled)
             (Shields shared MySQL/Mongo shards from thread pool exhaustion)
```

The Concurrent Request Limiter tracks active in-flight requests per merchant using atomic Redis counters or distributed lease tokens. Once a merchant exceeds their concurrency limit, further requests are rejected immediately, preventing noisy neighbors from exhausting database connections.

5. The Fail-Open Reliability Mandate

> **Fail-Open Axiom:** If the Redis rate-limiting cluster experiences network partitions, slow responses, or crashes, the rate limiter *must fail open*. Blocking live payment transactions due to an internal monitoring failure is unacceptable. Stripe defaults to letting traffic pass when rate-limiting telemetry is degraded.

Load Shedding Policies & Critical Path Protection

Source: Arpit Bhayani Dissection • Stripe Infrastructure
Focus: 20% Headroom, Worker Backpressure & Production Takeaways

6. Tier 3: Fleet-Level Load Shedding & Traffic Tiers

When total global traffic spikes (e.g. Cyber Monday), the fleet-level load shedder prioritizes traffic based on business criticality:

Traffic Category	Sample Endpoints	Capacity Policy	Degradation Behavior
Critical Path (Payments)	`POST /v1/charges`, `POST /v1/payment_intents`	**Guaranteed 20%+ reserved buffer**	Never shed until fleet reaches 100% hard capacity
Transactional Reads	`GET /v1/charges/ch_...`	Served normally under standard load	Shed only under severe cluster strain (>90% CPU)
Non-Critical Analytics	Dashboard charts, CSV exports, webhook retries	Operates on opportunistic surplus capacity	**Shed immediately when fleet load exceeds 80%**

> **The 20% Reservation Rule:** Stripe guarantees that live credit card checkouts always have at least 20% unallocated headroom. If total cluster utilization reaches 80%, background reporting is shed to prevent checkout payments from ever queuing.

7. Tier 4: Worker-Level Local Backpressure

The final defense operates on each individual application worker:

- **Local Metric Evaluation:** Tracks local process thread pool backlog (e.g. Puma queue latency) and local memory saturation.
- **Immediate Load Shedding:** If a specific worker instance experiences a localized GC pause or thread stall, it sheds non-critical incoming requests locally with HTTP 503 rather than queuing them and crashing.

8. First-Principles System Design Takeaways

Architecture Dimension	Naive Design Pattern	Stripe Production Standard
Rate Limiting Scope	Single global requests-per-minute counter	Decoupled 4-tier pipeline: RPS, concurrency, fleet shedder, worker shedder
Cost Metric	Treats all requests as cost = 1	Separate volume limits from expensive long-running query concurrency
Overload Handling	Drops all traffic uniformly or crashes	Classifies by business value; preserves 20% headroom for payment writes
Failure Behavior	Fail-closed (crashes block user traffic)	Fail-open (rate limiter failures permit payment requests)